BURSA TEKNİK ÜNİVERSİTESİ

GHOST OF WESTEROS

BİLAL DOĞRU 23360859071

İÇERİK

1.GİRİŞ …. 3

2.SİSTEM MİMARİSİ VE VERİ YAPILARI…. 4

2.1. Dinamik Envanter Yönetimi (Linked List / Bağlı Liste) …. 4
2.2. Zindan ve Geri İzleme Algoritması (Stack / Yığın) …. 5
2.3. Sıralı Görev Sistemi (Queue / Kuyruk) …. 5

3.OYUN MEKANİKLERİ VE ALGORİTMALAR…. 7

3.1. Refleks Tabanlı Savaş Sistemi…. 7
3.2. Hayatta Kalma ve Durum Yönetimi…. 8
3.3. Dosya İşlemleri (Save/Load Sistemi) …. 8
3.4. Atmosferik Metin İşleme…. 9

4. SENARYO VE DÜNYA TASARIMI…. 10

5. SONUÇ VE KAZANIMLAR…. 11

6. EKSİKLİKLER…. 12

7. KAYNAKÇA…. 13

1.GİRİŞ

"Ghost of Westeros" bir metin tabanlı rol yapma oyunu (RPG) türündedir. Aynı zamanda içinde hayatta kalma, strateji ve hikâye bazlı ilerleme öğeleri barındırır. Oyuncunun kararlarının hikâyeye ve karakterin gelişimine doğrudan etkisi olan bir deneyim sunar.

Bu proje, Westeros' un karanlık ve esrarengiz dünyasında geçen bir oyunun hikâyesini anlatmaktadır. Oyuncular, suikast yapma konusunda usta aynı zamanda usta bir çalgıcı olan bir kahramanın yerine geçerek bu dünya içinde hem heyecan verici hem de düşünmeye sevk eden bir maceraya atılacaklardır.

Hikâye, Westeros'ta kendi adaletini arayan bir suikastçıyı konu alır. Oyuncu, bu karakterin dünyasına adım atarak kendini zorlu kararların içinde bulacaktır. Her bir karar, kahramanın hikâyesine yön verecek ve oyunun kaderini belirleyecektir. Oyuncunun aldığı kararlar dünya üzerindi etkilerini hissettirecektir. Bu oyunda strateji başarı için oldukça önemli bir konudur.

Bu oyunun tüm geliştirme süreci Bilal Doğru tarafından gerçekleştirildi. Bilal Doğru Bursa Teknik Üniversitesi Bilgisayar Mühendisliği 1.sınıf öğrencisidir. Hikâyesinden oyun mekaniklerinden, görsel detaylara kadar her ayrıntısı Bilal Doğru tarafından tasarlandı ve uygulandı. Bu süreçte hem teknik hem de yaratıcı yönlerini kullanarak oyunun benzersiz bir yapıda olmasını sağlamaya çalıştı. Oyunun amacı, aksiyon dolu sahneler sunmak değil, oyuncuları derin bir hikâye ve karmaşık stratejilerle buluşturmaktır.

2.SİSTEM MİMARİSİ VE VERİ YAPILARI

Bu proje, C programlama dilinin sunduğu doğrudan bellek yönetimi yetenekleri kullanılarak geliştirilmiştir. Projenin temel mimarisi, statik ve sınırlı kapasiteye sahip diziler yerine, çalışma zamanında ihtiyaca göre büyüyüp küçülebilen dinamik veri yapıları üzerine inşa edilmiştir. Bu yaklaşım, sistem kaynaklarının verimli kullanılmasını sağlarken, oyunun genişletilebilirliğini ve performansını artırmaktadır. Veriler "Heap" bellek bölgesinde dinamik olarak tahsis edilmekte ve kullanım ömrü bittiğinde serbest bırakılarak bellek sızıntıları önlenmektedir.

2.1. Dinamik Envanter Yönetimi (Linked List / Bağlı Liste)

Oyun içerisinde karakterin satın aldığı silahlar, topladığı ganimetler ve kazandığı iksirler, Tek Yönlü Bağlı Liste (Singly Linked List) veri yapısı kullanılarak yönetilmektedir.

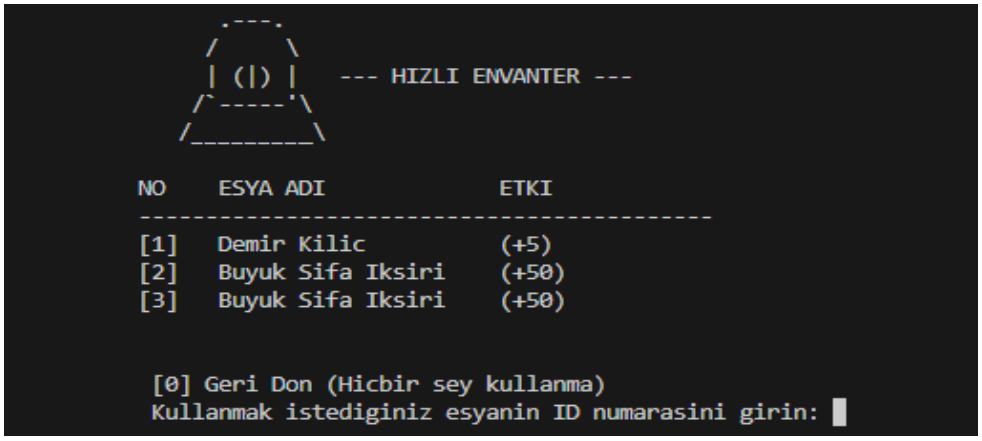
Teknik Yapı: Envanterdeki her bir nesne, bellekte bağımsız bir struct (yapı) olarak tutulur. Her düğüm (node), eşyaya ait verileri (ID, isim, hasar gücü, fiyat) ve kendisinden sonra gelen eşyanın bellek adresini tutan bir işaretçiyi (next pointer) barındırır.

Neden Tercih Edildi?

Dinamiklik: Bir RPG oyununda oyuncunun kaç eşya taşıyacağı önceden kestirilemez. Statik bir dizi (örn: Esya envanter [100]) tanımlamak, az eşya taşındığında bellek israfına, kapasite aşıldığında ise "Stack Overflow" veya veri kaybına yol açardı. Bağlı liste sayesinde envanter teorik olarak sınırsızdır.

Verimlilik: Envanterden bir iksir kullanıldığında veya bir eşya satıldığında, o eşyanın listeden silinmesi gerekir. Dizilerde silme işlemi kaydırma gerektirirken, bağlı listede sadece işaretçilerin (pointers) yönü değiştirilerek O(1) veya O(n) karmaşıklığında hızlıca silme işlemi yapılabilir.

Uygulama: Oyuncu mağazadan bir eşya satın aldığında envantere ekle() fonksiyonu çalışır, malloc ile bellekte yer ayrılır ve bu yeni düğüm listenin sonuna bağlanır.



(Şekil 2.1)

2.2. Zindan ve Geri İzleme Algoritması (Stack / Yığın)

Oyunun keşif modülü olan "Karanlık Zindan" , oyuncunun hareket geçmişini takip etmek ve labirentte kaybolmasını önlemek amacıyla Yığın (Stack) veri yapısı üzerine kurulmuştur. Bu yapı, LIFO (Last-In, First-Out / Son Giren İlk Çıkar) prensibiyle çalışır.

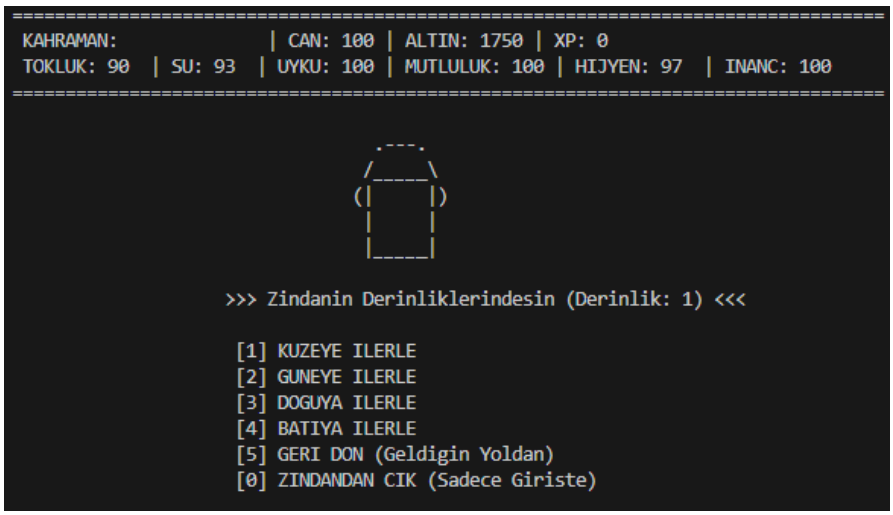
Teknik Yapı: Oyuncunun yaptığı her hareket (Kuzey, Güney, Doğu, Batı), bir düğüm olarak yığına eklenir. Yığının en tepesi (Top), oyuncunun attığı son adımı temsil eder.

Neden Tercih Edildi?

Geri İzleme (Backtracking): Labirent algoritmalarında çıkmaz sokağa giren bir algoritmanın geldiği yoldan geri dönmesi gerekir. Stack yapısı, doğası gereği "geri al" işlemleri için en uygun yapıdır. Oyuncu "Geri Dön" dediğinde, sistem en son yapılan hareketi hafızadan çağırır ve tersini uygular.

Hafıza Yönetimi: Oyuncu zindandan çıktığında, tüm hareket geçmişi pop() işlemiyle boşaltılarak bellek temizlenir.

Uygulama: Oyuncu 'Kuzey' yönüne gittiğinde push(KUZEY) fonksiyonu çalışır. Geri dönmek istediğinde pop() fonksiyonu çağrılır; eğer yığından 'Kuzey' çıkarsa, karakter otomatik olarak 'Güney' yönüne hareket ettirilir.



(Şekil 2.2)

2.3. Sıralı Görev Sistemi (Queue / Kuyruk)

Oyunun senaryo akışının tutarlı olması ve hikayenin belirlenen kronolojik sırada ilerlemesi için Kuyruk (Queue) veri yapısı kullanılmıştır. Bu yapı, FIFO (First-In, First-Out / İlk Giren İlk Çıkar) prensibiyle çalışır.

Teknik Yapı: Görevler, oyun başlatıldığında bir kuyruk yapısına sırasıyla eklenir (enqueue). Kuyruğun front (baş) işaretçisi aktif görevi, rear (son) işaretçisi ise eklenecek son görevi gösterir.

Neden Tercih Edildi?

Senaryo Bütünlüğü: Bir RPG oyununda oyuncunun 1. seviyede final boss ile savaşması veya hikayenin sonunu başından önce görmesi istenmez. Kuyruk yapısı, oyuncunun görevleri atlamasını engeller ve geliştiricinin kurguladığı sırayla ilerlemesini zorunlu kılar.

Uygulama: Oyuncu sadece kuyruğun en başındaki (front) görevi görebilir. Görev şartları (örneğin: "5. Seviye Ol") sağlandığında, gorev_tamamla() fonksiyonu çalışır. Bu fonksiyon dequeue işlemi yaparak tamamlanan görevi kuyruktan atar ve bir sonraki düğümü (görevi) kuyruğun başı yaparak aktif hale getirir.

```
==============================================================================
 KAHRAMAN:              | CAN: 100 | ALTIN: 1750 | XP: 0
 TOKLUK: 88  | SU: 90  | UYKU: 100 | MUTLULUK: 100 | HIJYEN: 97  | INANC: 100
==============================================================================

                                 _________________
                                / \               \
                               |   |  GOREVLER     |
                                \_ |               |
                                   |   PANOSU      |
                                   |               |
                                   |   _________   |
                                   |  /         \  |
                                   | |           | |
                                   \_|___________|_/

                  >>> AKTIF GOREV: Baslangic <<<
                  ------------------------------------------------
                  ACIKLAMA   : Westeros zorlu bir yer. Guclenmelisin.
                  GEREKSINIM : Seviye 2 Ol
                  ODUL       : 50 Altin | 20 XP | +1 Stat
                  ------------------------------------------------

                  Siradaki Gorev: ??? (Kilitli)
                  Kalan Toplam Gorev: Bir suru...


                   [1] GOREVI TAMAMLA (Kontrol Et)
                   [0] GERI DON
```

(Şekil 2.3)

3. OYUN MEKANİKLERİ VE ALGORİTMALAR

Bu bölümde, oyunun temel dinamiklerini oluşturan algoritmalar ve oyuncu deneyimini zenginleştiren teknik mekanikler detaylandırılmıştır. Standart metin tabanlı oyunların aksine, gerçek zamanlı sistemler ve dosya yönetimi gibi ileri seviye programlama teknikleri kullanılmıştır.

3.1. Refleks Tabanlı Gerçek Zamanlı Savaş Algoritması

Sıra tabanlı strateji oyunlarında genellikle statik bir bekleme süresi bulunur. Ancak bu projede, oyuncunun motor becerilerini ve tepki süresini oyunun bir parçası haline getiren dinamik bir savaş algoritması geliştirilmiştir.

Zaman Ölçüm Algoritması: C standart kütüphanesindeki clock() ve Windows API'ye ait GetTickCount() fonksiyonları kullanılarak milisaniye hassasiyetinde bir zamanlayıcı oluşturulmuştur.

baslangic = clock(); ile sayaç başlatılır.

scanf ile kullanıcıdan girdi beklenir.

bitis = clock(); ile sayaç durdurulur ve fark hesaplanır.

Hasar Hesaplama Mantığı:

Efsanevi Vuruş (< 1.0 sn): Eğer oyuncu rastgele üretilen (rand()) hedef harfe 1 saniyenin altında basarsa, hasar katsayısı x1.5 ile çarpılır.

Normal Vuruş (1.0 - 2.5 sn): Standart hasar uygulanır.

Iskalama (> 2.5 sn): Oyuncu geç kalırsa saldırı başarısız olur ve kendi kendine hasar verme (ceza) mekaniği devreye girer.

Savunma Mekaniği: Düşman saldırısı sırasında da aynı algoritma tersine çalışır. Doğru tuşa zamanında basmak, dayanıklılık statına bağlı olarak alınan hasarı minimize eder.

```
================================================================================
 KAHRAMAN:                | CAN: 66  | ALTIN: 1750 | XP: 0
 TOKLUK: 88  | SU: 90  | UYKU: 100 | MUTLULUK: 100 | HIJYEN: 97  | INANC: 100
================================================================================

                    ========================================
                              DUSMAN: Targaryen Askeri
                       Can: 47/70  |  Guc: 5  |  Ceviklik: 3
                    ========================================

                          [1] SALDIR (Risk Al)
                          [2] KAC (Tabanlari Yagla)
                          [3] CANTAYI AC (Iksir Ic)
```

(Şekil 3.1)

3.2. Hayatta Kalma ve Dinamik Durum Yönetimi

Oyun, sadece bir savaş simülasyonu değil, aynı zamanda bir kaynak yönetimi simülasyonudur. Oyuncunun kararlarını zorlaştırmak için Durum Makinesi benzeri bir yapı kurulmuştur.

Tokluk ve Susuzluk Döngüsü: Karakterin her hareketi (Zindan adımı, seyahat, savaş), belirli bir enerji maliyetine sahiptir. deger_azalt() fonksiyonu ile bu değişkenler sürekli güncellenir.

Kritik Seviye Kontrolü: uyari_fonksiyonu() adında özel bir denetleyici mekanizma geliştirilmiştir. Bu fonksiyon her tur çalışarak Tokluk veya Susuzluk değerlerinin 0'a ulaşıp ulaşmadığını kontrol eder. Eğer değer 0 ise, karakterin Can (HP) değeri her tur otomatik olarak düşürülür.

Etki: Bu mekanik, oyuncuyu "Zindana girmeden önce Han'a gidip yemek almalıyım" şeklinde stratejik planlama yapmaya zorlar.

3.3. Dosya İşlemleri (Save/Load Sistemi)

Oyunun sürekliliğini sağlamak ve oyuncunun ilerlemesini (seviye, para, konum vb.) korumak amacıyla Yapılandırılmış Metin Dosyası yöntemi kullanılmıştır. Bu yaklaşım, verilerin insan tarafından okunabilir olmasını sağlayarak hata ayıklama sürecini kolaylaştırmıştır.

Veri Yazma (Saving): Oyuncunun o anki durumu (struct Karakter içindeki değişkenler), fprintf fonksiyonu kullanılarak belirli bir format düzeninde .txt uzantılı bir metin dosyasına yazılır.

Veri Okuma (Loading): Oyun açılışında fopen ile dosyanın varlığı kontrol edilir. Eğer kayıt dosyası mevcutsa, fscanf fonksiyonu ile veriler sırasıyla okunur ve bellekteki oyuncu değişkenlerine (statlar, isim, para vb.) geri atanır.

Hata Yönetimi: Eğer dosya bulunamazsa veya bozuksa, sistem bunu algılar ve memset fonksiyonu ile belleği sıfırlayarak oyuncuyu otomatik olarak "Yeni Oyun" moduna yönlendirir.

Yeniden Doğuş (Respawn) Mekaniği: Karakter öldüğünde (can <= 0), oyun doğrudan sonlanmaz. Oyuncuya "İkinci Şans" sunulur.

Oyuncu kabul ederse; karakterin ismi ve çalgısı geçici değişkenlerde yedeklenir. memset ile tüm yapı sıfırlanır.

Yedeklenen veriler geri yüklenir ve karakter Seviye 1, 20 Altın ve tam sağlık ile başlangıç noktasına ışınlanır.

Bu sırada Görev Kuyruğu (Queue) ve Envanter (Linked List) bellek sızıntısını önlemek için temizlenir (free) ve yeniden başlatılır.

3.4. Atmosferik Metin İşleme (Typewriter Effect)

Oyunun hikaye anlatımını güçlendirmek için metinlerin ekrana **daktilo gibi harf harf** yazılmasını sağlayan özel bir görüntüleme algoritması geliştirilmiştir.

fflush(stdout) fonksiyonu kullanılarak çıktı tamponu her karakterden sonra temizlenir.

usleep() veya Sleep() fonksiyonları ile her harf arasına milisaniyelik gecikmeler eklenerek akıcı bir okuma deneyimi sağlanır. Bu özellik, özellikle oyunun giriş (Intro) bölümünde oyuncuyu atmosfere sokmak için kullanılmıştır.

4. SENARYO VE DÜNYA TASARIMI

"Ghost of Westeros", George R.R. Martin'in *Buz ve Ateşin Şarkısı* evreninde geçen karanlık bir RPG oyunudur. Oyuncu, Winterfell'den yola çıkan isimsiz bir savaşçıyı kontrol eder. Oyun dünyası, oyuncunun gelişimine göre tasarlanmış dört temel bölgeden oluşur:

Winterfell (Başlangıç): Oyuncunun temel mekanikleri öğrendiği ve maceraya adım attığı eğitim sahasıdır.

Han (Merkez): Oyunun ekonomi ve lojistik merkezidir. Oyuncu burada kazandığı altınlarla Demirci'den silah, Büyücü'den iksir satın alabilir ve Şifahane'de canını yenileyebilir.

Karanlık Zindan (Keşif): Yığın (Stack) veri yapısı üzerinde çalışan, yüksek risk ve ödül barındıran bir labirenttir. Oyuncu burada hem yönünü bulmaya çalışır hem de açlık/susuzluk gibi hayatta kalma (survival) mekanikleriyle mücadele eder.

Kral Toprakları (Final): Hikayenin sonlandığı ve en güçlü düşmanların bulunduğu ileri seviye bölgedir.

Oyunun atmosferini güçlendirmek için hikaye metinleri Daktilo Efekti ile harf ekrana yazdırılır. Ayrıca, oyuncu öldüğünde oyun tamamen bitmez; “Yeniden Doğuş” mekaniği sayesinde karakterin ismi korunarak, ancak fiziksel kazanımları sıfırlanarak maceraya tekrar başlama şansı tanınır.

5. SONUÇ VE KAZANIMLAR

"Ghost of Westeros" projesi, Bilgisayar Mühendisliği müfredatında yer alan "Algoritmalar ve Programlama" ile "Veri Yapıları" derslerinde edinilen teorik bilgilerin, somut ve kapsamlı bir mühendislik ürününe dönüştürülmüş halidir. Proje geliştirme sürecinde, eğitim boyunca öğrenilen temel ve ileri düzey programlama konseptlerinin tamamı aktif olarak kullanılmış ve pekiştirilmiştir.

Projenin sağladığı temel teknik kazanımlar şunlardır:

Veri Yapıları ve Algoritma Entegrasyonu:

Derslerde teorik olarak işlenen soyut veri tipleri oyunun ihtiyaçlarına göre en uygun yerlerde kullanılmıştır. Rastgele seçimler yerine; Envanter yönetimi için dinamikliği sağlayan Bağlı Liste (Linked List), Zindan keşfi ve geri alma işlemleri için Yığın (Stack), görev akışı için ise Kuyruk (Queue) yapıları başarıyla entegre edilmiştir. Bu sayede hangi veri yapısının hangi problem için daha verimli olduğu pratikte tecrübe edilmiştir.

İleri Seviye Bellek Yönetimi:

Statik dizilerin bellek kısıtlamaları aşılarak, İşaretçiler (Pointers) ve Dinamik Bellek Yönetimi konuları derinlemesine uygulanmıştır. malloc, calloc ve free fonksiyonları kullanılarak bellek sızıntıları önlenmiş, "Heap" ve "Stack" bellek bölgelerinin yönetimi konusunda yetkinlik kazanılmıştır.

Algoritmik Düşünce ve Karmaşıklık Yönetimi:

Oyunun savaş sistemindeki zamanlama algoritmaları, dosya okuma/yazma (File I/O) işlemleri ve karakterin durum yönetimi karmaşık problemlerin parçalara bölünerek çözülmesi yeteneğini geliştirmiştir.

Yazılım Mühendisliği Prensipleri:

Proje, tek bir dosya yerine Modüler Programlama tekniği ile .h (Header) ve .c (Source) dosyalarına ayrılarak geliştirilmiştir. Bu sayede kodun okunabilirliği, bakımı ve tekrar kullanılabilirliği artırılmıştır. Ayrıca proje süreci Git ve GitHub kullanılarak yönetilmiş, versiyon kontrol sistemleri üzerinde profesyonel bir deneyim elde edilmiştir.

Özetle; bu proje sadece çalışan bir oyun değil, aynı zamanda alınan eğitimin bir kanıtı niteliğindedir. Eksikliklere ve geliştirilebilir yanlara rağmen, hedeflenen mühendislik altyapısı başarıyla kurulmuştur.

6.EKSİKLİKLER VE GELECEK GELİŞTİRMELER

Bu proje geliştirilirken temel odak noktası, Veri Yapıları ve Algoritmalar dersinin kazanımlarını (Linked List, Stack, Queue, Pointer) pratik etmektir. Bu nedenle, projeye teknik katkı sağlamayacak, sadece kod tekrarına veya "veri girişine" sebep olacak eklentiler bilinçli olarak kapsam dışı bırakılmıştır.

Projenin mevcut haliyle optimize edildiği ve teknik gelişim açısından "doyum noktasına" ulaştığı alanlar şunlardır:

Grafiksel Kullanıcı Arayüzü (GUI): Oyun kasıtlı olarak Konsol Tabanlı bırakılmıştır. Unity veya Godot gibi hazır motorlar kullanmak, C dilinin sunduğu bellek yönetimi ve işaretçi (pointer) mantığının arka planda kalmasına neden olabilirdi. Projenin amacı görsellik değil, arka plandaki algoritmik mimariyi kurmaktır.

İçerik Çeşitliliği (Item/Enemy Spam): Oyuna yüzlerce farklı kılıç, zırh veya düşman eklenebilirdi. Ancak yazılımsal açıdan 5. kılıcı eklemek ile 500. kılıcı eklemek arasında algoritmik bir fark yoktur; bu sadece bir veri girişi işlemidir. Bu yüzden "Nicelik yerine Nitelik" prensibi benimsenmiş; sistemin *nasıl* çalıştığına odaklanılmıştır.

Veritabanı Entegrasyonu (SQL): Verilerin saklanması için SQL gibi karmaşık bir veritabanı yerine, Dosya İşlemleri (File I/O) tercih edilmiştir. Bu ölçekteki tek kişilik bir proje için harici bir veritabanı kurmak Gereğinden fazla mühendislik olacağından, C dilinin yerel dosya okuma/yazma yeteneklerini kullanmak daha öğretici bulunmuştur.

Yapay Zeka (AI) ve Yol Bulma: Oyunun haritası görsel bir 2D matris değil, soyut bir Yığın (Stack) yapısıdır. Bu nedenle A* (A-Star) gibi görsel yol bulma algoritmaları yerine, veri yapısının doğasına uygun olan Geri İzleme algoritması kullanılmıştır.

Sonuç olarak; projeye eklenecek her yeni özelliğin, öğrenciye yeni bir teknik kazanım sağlayıp sağlamadığı sorgulanmış ve sadece mühendislik becerisini geliştiren özellikler (Refleks Algoritması, Kuyruk Sistemi vb.) projeye dahil edilmiştir.

7.KAYNAKÇA

[1] Wikipedia yazarları. (n.d). *Game of Thrones*. Wikipedia. https://tr.wikipedia.org/wiki/Game_of_Thrones

[2] Explain It Simple. (n.d.). *Kumarın kökeni ve gelişimi*. Explain It Simple. https://www.explain-it-simple.com/tr/kumarin-kokeni-ve-gelisimi/